Lecture 7

Observational Data: Different Sampling Schemes

(Please, sign in on iClicker)

DSCI 554 - Experimentation and Causal Inference

Today's Learning Objectives

1. Introduce the use of different sampling schemes in observational studies.
2. Illustrate the concept of the proxy ground truth to assess different sampling schemes.
3. Apply complete simulation studies to a given sampling scheme via the proxy ground truth.
4. Explore the efficiency of the Case-Control sampling scheme via a modified power analysis.

Outline

1. Sampling Schemes in Observational Studies
2. The Western Collaborative Group Study Data
3. Running Three Sampling Schemes Using the Proxy Ground Truth
4. Sampling Scheme Assessment
5. Looking Further into the Case-Control Scheme

Previously...

We introduced observational studies to assess causality between a variable X and a response Y.
Moreover, we highlighted that observational studies are used when randomized experimentation is infeasible.
Hence, an observational study would try to make the best for data involving X, Y, and p confounders C_j ($i = 1, \ldots, p$).

Recap of `lecture6`

In `lecture6`, we reviewed a case study that followed two strategies:

1. Stratum-specific causal inference with multiple confounders.

We stratified our data according to the confounders C_j to see how X and Y are related.

> However, we encountered different estimation problems while following this approach.

Recap of `lecture6`

2. Overall Model-Based Causal Inference.

We pursued an overall model-based causal inference which is basically a regression of Y using X and $C_1, \ldots, C_p$ as regressors.

> This approach proved to be more effective since we controlled for the possible confounders in our model.

Recap of `lecture6`

Recall the overall model-based causal inference required three fundamental assumptions:

1. No interactions between strata, which implies a simple/smooth structure in how Y varies across strata.
2. No interactions involving X and any confounder C_j.
3. Have we captured all confounders in our selection of $C_1, \ldots, C_p$? A challenging assumption to fulfill!

From `causal analysis` to `causal design`

As we can see, we have already seen causal analysis in observational studies. It is time to check causal design in observational studies.

1. Sampling Schemes in Observational Studies

We will build on what we did in `lecture6` by looking at different sampling schemes for observational data.

> Heads-up: This is an important lecture since this is the first time we dig deeper into the field of statistical sampling.

There are different ways we can gather our data in observational studies.

Moreover

Mainly, we must explore how to select our sample before executing the study.
Our subsequent statistical analysis needs to account for how we sample our data.

1.1. Sampling Assessment Via the Ground Truth

Besides evaluating our statistical modelling (as we did via the three assumptions we explored in `lecture6`), we also need to assess any given sampling scheme before executing it.
Nonetheless, how are we supposed to evaluate these schemes?
We need to introduce a new concept to explain this matter. It is called the ground truth.

Heads-up

In Statistics and Machine Learning, the ground truth implicates checking the results of any given approach (e.g., a sampling scheme and its corresponding statistical model in the form of regression) in terms of its estimation accuracy against the real-world.
Nevertheless, assessing our methods in statistical inference is challenging because we do not have a concrete ground truth!

Under the frequentist paradigm

Since we DO NOT have access to the population data, we only rely on a sample which we use to estimate fixed population parameters.
Hence, the ground truth will not be available!
Not having access to the ground truth will be especially problematic if we want to infer something more complex than variable association, i.e., we would like to infer causality between X and Y.
However, we are now familiar with a powerful tool to build a proxy of the ground truth: **simulation**!

1.2. Simulation: A Proxy of the Ground Truth

We have been using basic Monte Carlo simulation throughout this block. However, we will use a more sophisticated simulation technique throughout this lecture.
The core ideas of this technique are the following:

1. Use a relevant sample dataset. This step assumes a previous sample dataset as representative enough of our population of interest. Ideally, we want our sampling methods to work on data with the typical characteristics of this population.

2. Fit a model. We will use the relevant sample dataset to fit a model of the response Y versus the variable X and a determined set of confounders C_j (as strata).
3. Simulate a proxy ground truth. This step will use the model from step 2 to simulate new outcomes Y.

   Basically, we would keep the same regressors' ranges since we are using their corresponding fitted model to simulate new observations. These are the typical ranges we would encounter in real population data.

iClicker Question

Answer TRUE or FALSE:

We could visualize the previous process of building a proxy ground truth as a modified bootstrapping taken to a higher level.

A. TRUE

B. FALSE

Answer

- It is TRUE.
- At the end of the day all models are just approximations of the real world. Thus, the best we can do is to rely on our available data to build a proxy of the ground truth so we can evaluate a potential future sampling scheme.
- Therefore, we could visualize the previous process as a modified bootstrapping taken to a higher level.

2. The Western Collaborative Group Study Data

We will retake the Western Collaborative Group Study (WCGS) data to illustrate the previously defined three-step process.
The WCGS data is a prospective cohort study that recruited middle-aged men between the ages of 39 and 59 who were employees of ten Californian companies.
The data was collected on 3154 subjects between 1960 and 1961.

2.1. Step 1: Use a Relevant Sample Dataset

We initially load our data. Then, we will eliminate those rows with missing data via `na.omit()` (missing data will again not be the scope of today's lecture!). We end up with $3,101$ subjects.
The three assumptions from `lecture6` will come into play in this part.

R Code Start Over Run Code

```r
wcgs <- as_tibble(read.dta("data/wcgs.dta"))
wcgs <- na.omit(wcgs)
wcgs
```

```
# A tibble: 3,101 × 22
     age arcus behpat   bmi chd69  chol   dbp dibpat height
id lnsbp lnwght

 1    50     1 A1      31.3 No      249    90 Type A     67
2343  4.88   5.30
 2    51     0 A1      25.3 No      194    74 Type A     73
3656  4.79   5.26
 3    59     1 A1      28.7 No      258    94 Type A     70
3526  5.06   5.30
 4    51     1 A1      22.1 No      173    80 Type A     69
22057  4.84   5.01
 5    44     0 A1      22.3 No      214    80 Type A     71
12927  4.84   5.08
 6    47     0 A1      27.1 No      206    76 Type A     64
16029  4.75   5.06
 7    40     0 A1      23.2 No      190    78 Type A     70
3894  4.80   5.09
```

```
 8    41     0 A1      23.0 No      212    84 Type A     70
11389  4.87   5.08
 9    50     1 A1      27.2 No      130    70 Type A     71
12681  4.72   5.27
10    43     0 A1      28.4 No      233    80 Type A     68
10005  4.79   5.23
# ℹ 3,091 more rows
# ℹ 10 more variables: ncigs , sbp , smoke , t1 ,
#   time169 , typchd69 , uni , weight , wghtcat ,
#   agec
```

Best model

Suppose the following model was the "best one" (note `age`, `bmi`, `smoke`, and `chol` are used as strata):

```
glm(chd69 ~ dibpat + age_bins + smoke + bmi_bins + chol_b
```

Select variables of interest

We select the following variables from `wcgs`:

- `dibpat`: Dichotomous behaviour pattern, a factor-type variable with two levels (`Type A` and `Type B`).
- `age`: Subject's age in years (a count-type variable).
- `bmi`: Subject's body mass index (BMI) in kg/m^2 (a continuous-type variable).
- `chol`: Subject's cholesterol levels in $\mathrm{mg}/100\ \mathrm{ml}$ (a continuous-type variable).
- `smoke`: Whether the subject smokes or not, a factor-type variable with two levels (`Yes` and `No`).
- `chd69`: Whether the subject experienced a coronary heart disease (CHD) event, a factor-type variable with two levels (`Yes` and `No`).

Code in R!

R Code Start Over Run Code

```
wcgs <- wcgs %>% select(dibpat, age, bmi, chol, smoke, chd69)
wcgs
```

```
# A tibble: 3,101 × 6
   dibpat   age   bmi  chol smoke chd69

 1 Type A    50  31.3   249 Yes   No
 2 Type A    51  25.3   194 Yes   No
 3 Type A    59  28.7   258 No    No
 4 Type A    51  22.1   173 No    No
 5 Type A    44  22.3   214 No    No
 6 Type A    47  27.1   206 Yes   No
 7 Type A    40  23.2   190 No    No
 8 Type A    41  23.0   212 Yes   No
 9 Type A    50  27.2   130 No    No
10 Type A    43  28.4   233 Yes   No
# ℹ 3,091 more rows
```

Main Statistical Inquiry

Recall the main statistical inquiry of this causal model:

> Does a `Type A` behaviour pattern (`dibpat`) LEAD to a coronary heart disease (CHD) event (`chd69`)?

From **Brand et al. (1976)**, the levels of `dibpat` are defined as follows:

> *Methods for the assessment of the dichotomous behavior pattern by a structured psychological interview in the WCGS have been described elsewhere. Type A behavior is characterized by enhanced aggressiveness and competitive drive, preoccupation with deadlines, and chronic impatience and sense of time urgency, in contrast with the more relaxed and less hurried Type B behavior pattern.*

Therefore

Y will be `chd69` and X will be `dibpat`, whereas `age`, `bmi`, `chol` and `smoke` will be confounders in the proxy of the ground truth.

Quartile strata

- Given the form of the model

```
glm(chd69 ~ dibpat + age_bins + smoke + bmi_bins + chol_b
```

  we need to create strata by confounder (except for `smoke` which is already categorical).

- These strata will be formed on a quartile basis.

Code in R!

R Code Start Over Run Code

```
wcgs <- wcgs %>% mutate(
    age_bins = cut(age, breaks = c(min(age), quantile(age, (1:3) / 4
    bmi_bins = cut(bmi, breaks = c(min(bmi), quantile(bmi, (1:3) / 4
    chol_bins = cut(chol, breaks = c(min(chol), quantile(chol, (1:3)
wcgs
```

```
# A tibble: 3,101 × 9
   dibpat   age   bmi  chol smoke chd69 age_bins bmi_bins
chol_bins

 1 Type A    50  31.3   249 Yes   No    (45,50]
(25.8,38.9] (223,253]
 2 Type A    51  25.3   194 Yes   No    (50,59]
(24.4,25.8] [103,197]
 3 Type A    59  28.7   258 No    No    (50,59]
(25.8,38.9] (253,645]
 4 Type A    51  22.1   173 No    No    (50,59]  [11.2,23]
[103,197]
 5 Type A    44  22.3   214 No    No    (42,45]  [11.2,23]
(197,223]
```

```
 6 Type A    47  27.1   206 Yes   No    (45,50]
(25.8,38.9] (197,223]
 7 Type A    40  23.2   190 No    No    [39,42]  (23,24.4]
[103,197]
 8 Type A    41  23.0   212 Yes   No    [39,42]  [11.2,23]
(197,223]
 9 Type A    50  27.2   130 No    No    (45,50]
(25.8,38.9] [103,197]
10 Type A    43  28.4   233 Yes   No    (42,45]
(25.8,38.9] (223,253]
# ℹ 3,091 more rows
```

Simulated population

- Once we have all variables ready, we construct a simulated population called `sim_pop`.
- This population is based on the fact that everyone in dataset `wcgs` contributes with twenty clones (in terms of X and C_js).
- We shuffle all rows at the end for subsequent sampling purposes.

> Heads-up: Note we are not using Y (`ch69`) to make these clones. We will construct Y in `step 2`.

Code in R!

R Code Start Over Run Code

```
sim_pop <- data.frame(
  dibpat = rep(wcgs$dibpat, each = 20),
  age_bins = rep(wcgs$age_bins, each = 20),
  smoke = rep(wcgs$smoke, each = 20),
  bmi_bins = rep(wcgs$bmi_bins, each = 20),
  chol_bins = rep(wcgs$chol_bins, each = 20)
)
```

Shuffling rows!

R Code Start Over Run Code

```r
set.seed(554) # Reproducibility
rows <- sample(nrow(sim_pop))
sim_pop <- sim_pop[rows, ]
head(sim_pop,n=10)
```

```
       dibpat age_bins smoke    bmi_bins chol_bins
46844 Type B  [39,42]    No (24.4,25.8] (197,223]
36954 Type B  (50,59]    No (25.8,38.9] [103,197]
47173 Type B  (42,45]    No   (23,24.4] [103,197]
44779 Type B  (50,59]    No   (23,24.4] (197,223]
2931  Type A  (50,59]   Yes   [11.2,23] (223,253]
3474  Type A  [39,42]   Yes   [11.2,23] (223,253]
347   Type A  (50,59]   Yes (24.4,25.8] (253,645]
41162 Type B  (50,59]   Yes (25.8,38.9] (253,645]
30223 Type A  (50,59]    No   (23,24.4] (223,253]
38595 Type B  (45,50]   Yes   [11.2,23] (253,645]
```

2.2. Step 2: Fit a Model

Now, using the sample in `wcgs`, we will fit a Binary Logistic regression as follows:

R Code Start Over Run Code

```
wcgs_bin_log_model <- glm(chd69 ~ dibpat + age_bins + smoke + bmi_bi
  family = "binomial", data =  wcgs)
```

Estimated regression coefficients

To mimic the variable structure of the actual dataset `wcgs` into `sim_pop`, we will use the estimated regression coefficients `wcgs_bin_log_model` to set the "true" relationship between the Y (`chd69`) and the X (`dibpat`) along with the confounders as strata.

R Code Start Over Run Code

```
wcgs_coef_pop <- coef(wcgs_bin_log_model)
round(wcgs_coef_pop, 2)
```

```
        (Intercept)        dibpatType A     age_bins(42,45]
 age_bins(45,50]
              -4.83                0.72                0.16
 0.63
    age_bins(50,59]            smokeYes   bmi_bins(23,24.4]
 bmi_bins(24.4,25.8]
               0.94                0.64                0.33
 0.48
 bmi_bins(25.8,38.9]  chol_bins(197,223]  chol_bins(223,253]
 chol_bins(253,645]
               0.68                0.35                0.88
```

```
1.29
```

2.3. Step 3: Simulate a Proxy Ground Truth

For the success event in the binary Y_i, we know how to compute predicted probabilities $\hat{\pi}_i$ given the estimated regression coefficients coming from a trained logistic regression model (such as `wcgs_bin_log_model`) for the ith subject:

$$\hat{\pi}_i = \frac{\exp\left[\text{logit}(\pi_i)\right]}{1 + \exp\left[\text{logit}(\pi_i)\right]}$$

where:

$$\text{logit}(\hat{\pi}_i) = \hat{\beta}_0 + \hat{\beta}_X X_{i,1} + \hat{\beta}_1 C_{i,1} + \ldots + \hat{\beta}_p C_{i,p}.$$

Our function in R!

The previous computation can be put in a function as:

R Code Start Over Run Code

```
pred_prob <- function(logit) {exp(logit) /(1 + exp(logit))}
```

Steps to get proxy ground truth

Using the estimated regression coefficients coming from `wcgs_bin_log_model` and stored in `coef_pop`, we will do the following in `sim_pop` to obtain the response Y as our proxy ground truth:

1. Obtain the design matrix (via the function `model.matrix()`) of `sim_pop`. A design matrix is also called a regressor matrix. Since we only have dummy variables, it will contain `0`s and `1`s.

R Code Start Over Run Code

```
model_matrix_sim_pop <- model.matrix(~ dibpat + age_bins + smoke + b
head(model_matrix_sim_pop, n=8)
```

```
      (Intercept) dibpatType A age_bins(42,45]
age_bins(45,50] age_bins(50,59]
46844           1             0                0
0               0
36954           1             0                0
0               1
47173           1             0                1
0               0
44779           1             0                0
0               1
2931            1             1                0
0               1
3474            1             1                0
0               0
```

```
347              1              1                0
0                1
41162            1              0                0
0                1
      smokeYes bmi_bins(23,24.4] bmi_bins(24.4,25.8]
bmi_bins(25.8,38.9]
46844        0                  0                   1
0
36954        0                  0                   0
1
47173        0                  1                   0
0
44779        0                  1                   0
0
2931         1                  0                   0
0
3474         1                  0                   0
0
347          1                  0                   1
0
41162        1                  0                   0
1
      chol_bins(197,223] chol_bins(223,253]
chol_bins(253,645]
46844                  1                  0
0
36954                  0                  0
```

```
0
47173                    0                    0
0
44779                    1                    0
0
2931                     0                    1
0
3474                     0                    1
0
347                      0                    0
1
41162                    0                    0
1
```

2. To obtain the proxy ground truth of Y (`chd69`) for each subject in `sim_pop`, we do the following:
   - Obtain their corresponding predicted probabilities using the estimated regression coefficients stored in `wcgs_coef_pop` and the `model_matrix_sim_pop`.
   - Then, these estimated probabilities will be the argument `prob` in the random number generator `rbinom()`. We use `rbinom()` (with `size = 1`) to add random noise to a Bernoulli trial (i.e., each binary response Y by subject in `sim_pop`).

R Code Start Over Run Code

```
set.seed(554) # Reproducibility
sim_pop$chd69 <- rbinom(n = nrow(sim_pop),
                        size = 1,
                        prob = pred_prob(logit = model_matrix_sim_po
```

R Code Start Over Run Code

```
sim_pop <- sim_pop %>% mutate(chd69 = as.factor(chd69))
head(sim_pop,n=12)
```

```
      dibpat age_bins smoke    bmi_bins chol_bins chd69
46844 Type B  [39,42]    No (24.4,25.8] (197,223]     0
36954 Type B  (50,59]    No (25.8,38.9] [103,197]     0
47173 Type B  (42,45]    No   (23,24.4] [103,197]     0
44779 Type B  (50,59]    No   (23,24.4] (197,223]     0
2931  Type A  (50,59]   Yes   [11.2,23] (223,253]     0
3474  Type A  [39,42]   Yes   [11.2,23] (223,253]     0
347   Type A  (50,59]   Yes (24.4,25.8] (253,645]     0
41162 Type B  (50,59]   Yes (25.8,38.9] (253,645]     0
30223 Type A  (50,59]    No   (23,24.4] (223,253]     0
38595 Type B  (45,50]   Yes   [11.2,23] (253,645]     0
50054 Type B  (50,59]   Yes   (23,24.4] (197,223]     0
24883 Type A  (42,45]    No (25.8,38.9] (223,253]     0
```

What just happened here?

Our simulated population `sim_pop` has the responses `chd69` as a proxy ground truth, which were generated using the estimated regression coefficients from the logistic model trained with the actual sample data `wcgs`.

Contingency table `dibpat` vs `chd69`

R Code Start Over Run Code

```r
table(sim_pop$dibpat, sim_pop$chd69)
```

```
            0     1
Type B 29203  1537
Type A 27800  3480
```

3. Running Three Sampling Schemes Using the Proxy Ground Truth

Previously, we built an artificial population `sim_pop` with a proxy ground truth in our response Y. It is now time to use this simulated population to compare three sampling schemes:

- Cross-sectional
- Case-control
- Cohort

Main Statistical Inquiry

If we want to run a new observational study, in terms of our proxy ground truth stored in `sim_pop`, what sampling scheme would be the most accurate and precise?

> Heads-up: For our sampling schemes comparisons, we will begin with a common sample size of $n = 1000$ participants.

R Code Start Over Run Code

```
n_sample <- 1000 # Sample size
N_pop <- nrow(sim_pop) # Population size
```

3.1. Cross-Sectional Sampling Scheme

- Cross-sectional (CS) is a contemporaneous scheme.
- It will reach the population and grab a simple random sample of size n (i.e., it will not consider any variable of interest when sampling).
- It is analogous to taking an instantaneous snapshot of all the study variables.

Sampling in R for CS!

R Code Start Over Run Code

```
set.seed(554) # Reproducibility

CS_sampled_subjects <- sample(1:N_pop, size = n_sample, replace = FA
CS_sample <- sim_pop[CS_sampled_subjects, ] # Collecting data from a
head(CS_sample,n=10)
```

```
       dibpat age_bins smoke    bmi_bins chol_bins chd69
9820   Type A  [39,42]    No (24.4,25.8] (197,223]     0
2158   Type A  [39,42]   Yes   (23,24.4] (197,223]     0
15091  Type A  (45,50]    No   [11.2,23] [103,197]     0
23570  Type A  (45,50]   Yes (24.4,25.8] [103,197]     0
52796  Type B  [39,42]   Yes (24.4,25.8] [103,197]     0
8114   Type A  (50,59]   Yes   [11.2,23] (197,223]     0
35534  Type B  (42,45]    No (25.8,38.9] [103,197]     0
4538   Type A  (42,45]   Yes   (23,24.4] (223,253]     1
59739  Type B  [39,42]    No (25.8,38.9] (223,253]     0
48872  Type B  [39,42]    No (24.4,25.8] [103,197]     0
```

3.2. Case-Control Sampling Scheme

Case-control (CC) is a retrospective scheme. It will sample a group of subjects with the study's response success ($Y = 1$) and a group without ($Y = 0$).
Half of the overall sample will correspond to subjects with $Y = 1$ and the other half to $Y = 0$.
The researcher will ask each subject: *have you ever been exposed to the condition X in the past ($X = 0$ never and $X = 1$ ever).*

> Heads-up: In this scheme, the case group is $Y = 1$ and the control group is $Y = 0$.

Sampling in R for CC!

R Code Start Over Run Code

```
set.seed(554) # Reproducibility

CC_sampled_subjects <- c(sample((1:N_pop)[sim_pop$chd69 == "0"], siz
  sample((1:N_pop)[sim_pop$chd69 == "1"], size = n_sample / 2, repla
)
CC_sample <- sim_pop[CC_sampled_subjects, ] # Collecting data from a
head(CC_sample,n=10)
```

```
      dibpat age_bins smoke    bmi_bins chol_bins chd69
37462 Type B  (50,59]   Yes   [11.2,23] (223,253]     0
20335 Type A  (45,50]   Yes   (23,24.4] (197,223]     0
47457 Type B  (42,45]   Yes   [11.2,23] [103,197]     0
46667 Type B  (45,50]    No (25.8,38.9] (253,645]     0
36081 Type B  [39,42]   Yes   (23,24.4] (223,253]     0
37318 Type B  (50,59]    No (25.8,38.9] (223,253]     0
31184 Type A  [39,42]    No (24.4,25.8] (197,223]     0
49847 Type B  (50,59]   Yes   [11.2,23] (253,645]     0
27820 Type A  (45,50]    No   (23,24.4] (223,253]     0
1186  Type A  [39,42]   Yes (25.8,38.9] (197,223]     0
```

DSCI 554 - Experimentation and Causal Inference

3.3. Cohort Sampling Scheme

Cohort (CO) is a prospective scheme. It will sample a group n of subjects who show $Y = 0$ when the study begins.
Half of the overall sample will correspond to subjects currently without exposure (i.e., $X = 0$) and the half with exposure (i.e., $X = 1$).
The researcher will keep in touch with each subject for a given amount of time. At the end of the study, the researcher will check who ended with $Y = 0$ or $Y = 1$.

Heads-up

In this sampling scheme, using our proxy ground truth, the column `chd69` in `CO_sampled_subjects` is assumed as the recorded outcome at the end of the time frame.

R Code Start Over Run Code

```r
set.seed(554) # Reproducibility

CO_sampled_subjects <- c(
  sample((1:N_pop)[sim_pop$dibpat == "Type B"],
    size = n_sample / 2, replace = FALSE
  ),
  sample((1:N_pop)[sim_pop$dibpat == "Type A"],
    size = n_sample / 2, replace = FALSE
  )
)
CO_sample <- sim_pop[CO_sampled_subjects, ]
head(CO_sample)
```

```
       dibpat age_bins smoke    bmi_bins chol_bins chd69
59228 Type B  (45,50]   Yes   [11.2,23] (197,223]     0
56210 Type B  [39,42]    No   (23,24.4] [103,197]     0
61571 Type B  (45,50]    No   (23,24.4] (197,223]     0
46283 Type B  [39,42]    No (24.4,25.8] (223,253]     0
38560 Type B  (45,50]    No   [11.2,23] (253,645]     0
48247 Type B  (50,59]    No   (23,24.4] [103,197]     0
```

DSCI 554 - Experimentation and Causal Inference

3.4. When to Use What?

This is an important question regarding CS, CC, and CO studies. In general, these are the guidelines:

- CS studies are ideal in early research stages.
  - This is because they are faster to run and offer good preliminary insights of causality between X and Y, before conducting more complex longitudinal studies such as CO studies (i.e., studies that require following up patients over time).

- CC is ideal in situations where the outcome $Y = 1$ is rare in the population of interest; i.e., we will not eventually have too many cases $Y = 1$ if we follow up the patients over time as in a CO.
  - Hence, we recruit a considerable number of patients with $Y = 1$ for our CC study.
- Compared to CC, CO studies are ideal in cases where the exposure $X = 1$ is rare in the population of interest; i.e, we will not have too many cases $X = 1$ if we implement a CC study by just recruiting as many patients as possible with $Y = 1$.

Thus

For CO, we recruit a considerable number of patients with $X = 1$ at the beginning of the study and follow up their outcome status Y over time.

4. Sampling Scheme Assessment

Recall that the estimated regression coefficients we used to obtain our proxy ground truth were the ones below.
These coefficients were obtained using the real data stored in `wcgs`.

R Code Start Over Run Code

```
round(wcgs_coef_pop, 2)
```

```
        (Intercept)        dibpatType A     age_bins(42,45]
age_bins(45,50]
              -4.83                0.72                0.16
0.63
    age_bins(50,59]            smokeYes   bmi_bins(23,24.4]
bmi_bins(24.4,25.8]
               0.94                0.64                0.33
0.48
bmi_bins(25.8,38.9]  chol_bins(197,223]  chol_bins(223,253]
chol_bins(253,645]
               0.68                0.35                0.88
1.29
```

DSCI 554 - Experimentation and Causal Inference

Log-OR for the variable `dipat`

Note that the log of the odds ratio (log-OR) for the variable `dipat` is 0.72.
We used this estimated log-OR to build the outcomes in our proxy ground truth (i.e., `chd69` in `sim_pop`).
Hence, we can evaluate how accurate the estimations are via `CS_sample`, `CC_sample`, and `CO_sample`.

4.1 Fitting the Models Using the Three Sampling Schemes

We now proceed to fit the corresponding regression models with our three different sampling schemes.
The summaries are also shown for our target X (i.e., `dibpat`) along with their 95% confidence intervals (CIs).

Summary of CS!

R Code Start Over Run Code

```
CS_bin_log_model <- glm(chd69 ~ dibpat + age_bins + smoke + bmi_bins
                        family = "binomial", data = CS_sample)
summary_CS <- tidy(CS_bin_log_model, conf.int = TRUE) %>% mutate_if(
summary_CS[2, ]
```

```
# A tibble: 1 × 7
  term         estimate std.error statistic p.value
conf.low conf.high

1 dibpatType A    0.834     0.249      3.35   0.001
0.357      1.34
```

Summary of CC!

R Code Start Over Run Code

```r
CC_bin_log_model <- glm(chd69 ~ dibpat + age_bins + smoke + bmi_bins
                        family = "binomial", data = CC_sample)
summary_CC <- tidy(CC_bin_log_model, conf.int = TRUE) %>% mutate_if(
summary_CC[2, ]
```

```
# A tibble: 1 × 7
  term         estimate std.error statistic p.value
conf.low conf.high

1 dibpatType A    0.942     0.141      6.68       0
0.667      1.22
```

Summary of CO!

R Code Start Over Run Code

```
CO_bin_log_model <- glm(chd69 ~ dibpat + age_bins + smoke + bmi_bins
                        family = "binomial", data = CO_sample)
summary_CO <- tidy(CO_bin_log_model, conf.int = TRUE) %>% mutate_if(
summary_CO[2, ]
```

```
# A tibble: 1 × 7
  term         estimate std.error statistic p.value
conf.low conf.high

1 dibpatType A    0.262     0.246      1.06   0.287
-0.217     0.751
```

Using these SINGLE replicates by sampling scheme, in terms of our main statistical inquiry, we highlight the following:

- Both CS and CC schemes provide significant and accurate estimates of X (compared to the value used to obtain our proxy ground truth, 0.72). Nonetheless, CC shows a more precise estimate (i.e., a narrower 95% CI).
- On the other hand, CO does not even provide a significant estimate!

Nevertheless

Recall these are just three replicates! We need to run a more comprehensive simulation study (as we did in previous lectures).

4.2 Running Simulation Studies Using the Three Sampling Schemes

The previous results are merely single replicates of any given simulation study for each sampling scheme.
The function `sim_study()` draws the corresponding samples by scheme. Then, it runs the corresponding regression models.

Arguments for our `sim_study()` function!

It needs the following arguments:

- `pop_data`: The population data from which we will draw the sample.
- `n`: The sample size by scheme.
- `alpha`: Significance level α.
- `log_OR`: The "true" log-OR each scheme is willing to estimate.
- `num_replicates`: The number of replicates by scheme.

`sim_study()` function in R!

R Code Start Over Run Code

```r
sim_study <- function(pop_data, n, alpha, log_OR, num_replicates) {
  res <- list(NULL) # Setting up matrix with metrics
  res[[1]] <- res[[2]] <- res[[3]] <- matrix(NA, num_replicates, 3)

  suppressMessages(for (lp in 1:num_replicates) { # Otherwise, we g
    # Obtaining samples by scheme
    # CS
    CS_sampled_subjects <- sample(1:nrow(pop_data), size = n, repla
    CS_sample <- pop_data[CS_sampled_subjects, ]
    # CC
    CC_sampled_subjects <- c(
      sample((1:nrow(pop_data))[pop_data$chd69 == "0"],
        size = n / 2, replace = F
      ),
      sample((1:nrow(pop_data))[pop_data$chd69 == "1"],
        size = n / 2, replace = F
      )
    )
    CC_sample <- pop_data[CC_sampled_subjects, ]
    # CO
    CO_sampled_subjects <- c(
      sample((1:nrow(pop_data))[pop_data$dibpat == "Type B"],
        size = n / 2, replace = F
      ),
      sample((1:nrow(pop_data))[pop_data$dibpat == "Type A"],
        size = n / 2, replace = F
      )
```

```
    )
    CO_sample <- pop_data[CO_sampled_subjects, ]

    # Do the three analyses
    # CS
    CS_bin_log_model <- glm(chd69 ~ dibpat + age_bins + smoke + bmi
      family = "binomial", data = CS_sample
    )
    # CC
    CC_bin_log_model <- glm(chd69 ~ dibpat + age_bins + smoke + bmi
      family = "binomial", data = CC_sample
    )
    # CO
    CO_bin_log_model <- glm(chd69 ~ dibpat + age_bins + smoke + bmi
      family = "binomial", data = CO_sample
    )

    # and the takeaways
    res[[1]][lp, ] <- c(coef(CS_bin_log_model)["dibpatType A"], con
    res[[2]][lp, ] <- c(coef(CC_bin_log_model)["dibpatType A"], con
    res[[3]][lp, ] <- c(coef(CO_bin_log_model)["dibpatType A"], con
  })

  # Summaries
  BIAS <- sapply(
    res,
    function(mat) {
```

```
      mean(mat[, 1]) - log_OR
    }
  )
  vrnc <- sapply(res, function(mat) {
    var(mat[, 1])
  })
  CVRG <- sapply(res,
    function(mat, trg) {
      mean((mat[, 2] < trg) & (trg < mat[, 3]))
    },
    trg = log_OR
  )
  PWR <- sapply(res, function(mat) {
    mean(mat[, 2] > 0)
  })
  RMSE <- sqrt(BIAS^2 + vrnc)

  opt <- cbind(BIAS, RMSE, CVRG, PWR)
  rownames(opt) <- c("Cross-Sectional (CS)", "Case-Control (CC)", "

  return(opt)
}
```

Four metrics are back!

The function returns the following metrics:

- BIAS: The average, over the `num_replicates`, of $\widehat{\log\text{-OR}} - \log\text{-OR}$. The larger the average, the less accuracy.
- ROOT-MEAN-SQUARED ERROR (RMSE): The square root of the average (over the `num_replicates`) of $(\widehat{\log\text{-OR}} - \log\text{-OR})^2$. It has the following characteristics:
  - Interpretable as the typical magnitude of estimation error.
  - Captures bias and variance involved in the procedure.
- COVERAGE (CVRG): The proportion (among the `num_replicates`) of the 95% CIs that contain the "true" $\log\text{-OR}$.
- POWER (PWR): It is applicable when $\log\text{-OR} \neq 0$. The proportion of the `num_replicates` CIs that exclude zero.

Run the simulations!

We run the simulation studies with $n = 1000, 1500, 2000$.

```
set.seed(554) # Reproducibility
sim_study_n_1000 <- sim_study(
  pop_data = sim_pop, n = 1000, alpha = 0.05,
  log_OR = wcgs_coef_pop["dibpatType A"], num_replicates = 500
)
```

```
set.seed(554) # Reproducibility
sim_study_n_1500 <- sim_study(
  pop_data = sim_pop, n = 1500, alpha = 0.05,
  log_OR = wcgs_coef_pop["dibpatType A"], num_replicates = 500
)
```

```
set.seed(554) # Reproducibility
sim_study_n_2000 <- sim_study(
  pop_data = sim_pop, n = 2000, alpha = 0.05,
  log_OR = wcgs_coef_pop["dibpatType A"], num_replicates = 500
)
```

Performance Metrics

```
round(sim_study_n_1000, 4)
```

A matrix: 3 × 4 of type dbl

	BIAS	RMSE	CVRG	PWR
Cross-Sectional (CS)	0.0674	0.2740	0.956	0.860
Case-Control (CC)	0.0329	0.1385	0.954	1.000
Cohort (CO)	0.0573	0.2657	0.944	0.866

```
round(sim_study_n_1500, 4)
```

A matrix: 3 × 4 of type dbl

	BIAS	RMSE	CVRG	PWR
Cross-Sectional (CS)	0.0455	0.2173	0.948	0.964
Case-Control (CC)	0.0239	0.1110	0.960	1.000
Cohort (CO)	0.0361	0.2047	0.966	0.972

```
round(sim_study_n_2000, 4)
```

A matrix: 3 × 4 of type dbl

	BIAS	RMSE	CVRG	PWR
Cross-Sectional (CS)	0.0391	0.1883	0.944	0.994
Case-Control (CC)	0.0194	0.1010	0.952	1.000
Cohort (CO)	0.0269	0.1842	0.954	0.992

In-class Question

What is the conclusion by looking at these metrics by sampling scheme?

Answer

Starting $n = 1000$, CC shows the highest power among the three schemes. Moreover, it has the smallest average bias and RMSE. This pattern remains in $n = 1500, 2000$.
Even though the other two schemes, CS and CO, improve as we increase the sample size, they equally underperform compared to CC.
Therefore, to answer our main statistical inquiry, CC has the best performance across the three sampling schemes with a smaller sample size.

5. Looking Further into the Case-Control Scheme

- Our previous simulation studies indicate that CC performs better when using our proxy ground truth to assess each sampling scheme.
- Nonetheless, we have an artificial population of $N = 62,020$ subjects where $1,537 + 3,480 = 5,017$ subjects experience (or will experience!) a CHD event:

Contingency Table

R Code Start Over Run Code

```r
table(sim_pop$dibpat, sim_pop$chd69)
```

```
            0     1
Type B 29203  1537
Type A 27800  3480
```

Compute Probability

In this case, we can see that only $\frac{1{,}537+3{,}480}{62{,}020} \times 100\% = 8.09\%$ of the N subjects experience the "success" event in Y.
Therefore, this event can be classified as relatively rare.

Furthermore

Recall the CC definition:

> Case-control (CC) is a retrospective scheme. It will sample a group of subjects with the study's response success ($Y = 1$) and a group without ($Y = 0$). Half of the overall sample will correspond to subjects with $Y = 1$ and the other half to $Y = 0$. The researcher will ask each subject: *have you ever been exposed to the condition X in the past ($X = 0$ never and $X = 1$ ever).*

Over/Under-sampling problem

Given the CC's nature, and the population's situation, this sampling scheme is clearly oversampling cases ($Y = 1$) and undersampling controls ($Y = 0$). This matter will pave the way to a secondary statistical inquiry.

Secondary Statistical Inquiry

Using a CC sampling scheme, is oversampling cases and undersampling controls a winning strategy in terms of precision?
This statistical inquiry implies a Modified Power Analysis but not with the overall sample size n.
We will use the $\mathrm{Case{:}Control\ ratio}$ instead.

This ratio behaves as follows:

- $\text{Case:Control ratio} = 1$ indicates we have the same number of cases and controls in our sample of size n.
- $\text{Case:Control ratio} > 1$ indicates we have more cases than controls in our sample of size n.
- $\text{Case:Control ratio} < 1$ indicates we have less cases than controls in our sample of size n.

Modified Power Analysis

This Modified Power Analysis will not directly involve the power of the test but the standard error (SE) of the estimate of the log-OR. The standard error is a crucial component of a CI; the smaller the SE, the more precise our estimate is.

Before explaining the components of our Modified Power Analysis, let us review the components of the contingency table.

Review

Suppose both X and Y are binary variables. Then, the absolute frequencies by each combination of factor levels can be put in a contingency table as follows:

	$Y = 0$	$Y = 1$
$X = 0$	$n_{X=0,Y=0}$	$n_{X=0,Y=1}$
$X = 1$	$n_{X=1,Y=0}$	$n_{X=1,Y=1}$

Furthermore let:

$n_{Y=1} = n_{X=0,Y=1} + n_{X=1,Y=1} \quad \text{and}$

$n_{Y=0} = n_{X=0,Y=0} + n_{X=1,Y=0}$

be the respective numbers of cases and controls in the sample of size n.

We know that

$$\begin{aligned}\widehat{\log\text{-OR}} &= \log(n_{X=1,Y=1}) - \log(n_{X=1,Y=0}) \\ &\quad - \left[\log(n_{X=0,Y=1}) - \log(n_{X=0,Y=0})\right] \\ \widehat{\log\text{-OR}} &= \underbrace{\log(n_{X=1,Y=1}) - \log(n_{X=0,Y=1})}_{\text{Cases}} \\ &\quad - \underbrace{\left[\log(n_{X=1,Y=0}) - \log(n_{X=0,Y=0})\right]}_{\text{Controls}}\end{aligned}$$

Then

We rearrange the expression above using logarithmic properties:

$$
\begin{aligned}
\log\text{-}\hat{\mathrm{OR}} &= \log\left(\frac{n_{X=1,Y=1}}{n_{X=0,Y=1}}\right) - \log\left(\frac{n_{X=1,Y=0}}{n_{X=0,Y=0}}\right) \\
&= \log\left(\frac{n_{X=1,Y=1}/n_{Y=1}}{n_{X=0,Y=1}/n_{Y=1}}\right) - \log\left(\frac{n_{X=1,Y=0}/n_{Y=0}}{n_{X=0,Y=0}/n_{Y=0}}\right) \\
&= \log\left(\frac{n_{X=1,Y=1}}{n_{Y=1}}\right) - \log\left(\frac{n_{X=0,Y=1}}{n_{Y=1}}\right) \\
&- \left[\log\left(\frac{n_{X=1,Y=0}}{n_{Y=0}}\right) - \log\left(\frac{n_{X=0,Y=0}}{n_{Y=0}}\right)\right]
\end{aligned}
$$

$$
\begin{aligned}
\widehat{\text{log-OR}} &= \log\left(\frac{n_{X=1,Y=1}}{n_{Y=1}}\right) - \log\left(1 - \frac{n_{X=1,Y=1}}{n_{Y=1}}\right) \\
&\quad - \left[\log\left(\frac{n_{X=1,Y=0}}{n_{Y=0}}\right) - \log\left(1 - \frac{n_{X=1,Y=0}}{n_{Y=0}}\right)\right] \\
\widehat{\text{log-OR}} &= \log(p_{X=1,Y=1}) - \log(1 - p_{X=1,Y=1}) \\
&\quad - \left[\log(p_{X=1,Y=0}) - \log(1 - p_{X=1,Y=0})\right]
\end{aligned}
$$

Proportion of Cases `p_effects`

- The previous arrangements set up the $\log$-OR in terms of the proportion of cases $p_{X=1,Y=1}$ and $p_{X=1,Y=0}$ with respect to $n_{Y=1}$ and $n_{Y=0}$.
- We name the previous proportions as effects.
- We will check the following combinations in our modified power analysis.

`p_effects` values in R!

R Code Start Over Run Code

```r
p_effects <- matrix(0.0,4,2)
p_effects[1,] <- c(0.25,0.3)
p_effects[2,] <- c(0.25,0.4)
p_effects[3,] <- c(0.25,0.5)
p_effects[4,] <- c(0.25,0.6)
colnames(p_effects) <- c("p_X1_Y0", "p_X1_Y1")
p_effects
```

```
     p_X1_Y0 p_X1_Y1
[1,]    0.25     0.3
[2,]    0.25     0.4
[3,]    0.25     0.5
[4,]    0.25     0.6
```

Note we gradually increase the $p_{X=1,Y=1}$ while keeping $p_{X=1,Y=0}$ constant and smaller.

log-OR Estimates

The matrix `p_effects` and our rearranged expression for $\log\text{-OR}$ yield the following estimates:

R Code Start Over Run Code

```
log_OR <- log(p_effects[, 2]) - log(1 - p_effects[, 2]) -
  (log(p_effects[, 1]) - log(1 - p_effects[, 1]))
round(log_OR, 2)
```

```
[1] 0.25 0.69 1.10 1.50
```

Since we are using the SE of the $\log\text{-OR}$ as our key metric, let us review its form (along with its R function):

$$\text{SE} = \sqrt{\frac{1}{n_{X=1,Y=1}} + \frac{1}{n_{X=1,Y=0}} + \frac{1}{n_{X=0,Y=1}} + \frac{1}{n_{X=0,Y=0}}}$$

R Code Start Over Run Code

```r
logOR_SE <- function(table) {
  sqrt(sum(1 / table))
}
```

- We actually need to compute subsample sizes by cell in the contingency table.
- We will set up a constant number of controls $n_{Y=0} = 500$ and an increasing number of cases from $n_{Y=1} = 25$ to 750.
- This sequence eventually will EVENTUALLY reflect an oversampling of cases and undersampling of controls.

R Code Start Over Run Code

```r
n_controls <- 500
n_cases <- seq(from = 25, to = 750, by = 25)
n_cases
```

```
 [1]  25  50  75 100 125 150 175 200 225 250 275 300 325
350 375 400 425 450 475
[20] 500 525 550 575 600 625 650 675 700 725 750
```

The matrix `SE_log_OR` will store all the SEs by each value of the estimated $\log$-OR in `log_OR` and number of cases $n_{Y=1}$ in `n_cases`.

R Code Start Over Run Code

```
SE_log_OR <- matrix(0.0, dim(p_effects)[1], length(n_cases))
SE_log_OR
```

```
     [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[,11] [,12] [,13] [,14]
[1,]    0    0    0    0    0    0    0    0    0     0
0     0     0     0
[2,]    0    0    0    0    0    0    0    0    0     0
0     0     0     0
[3,]    0    0    0    0    0    0    0    0    0     0
0     0     0     0
[4,]    0    0    0    0    0    0    0    0    0     0
0     0     0     0
     [,15] [,16] [,17] [,18] [,19] [,20] [,21] [,22] [,23]
[,24] [,25] [,26]
[1,]     0     0     0     0     0     0     0     0     0
0     0     0
[2,]     0     0     0     0     0     0     0     0     0
```

```
0     0     0
[3,]     0     0     0     0     0     0     0     0     0
0     0     0
[4,]     0     0     0     0     0     0     0     0     0
0     0     0
     [,27] [,28] [,29] [,30]
[1,]     0     0     0     0
[2,]     0     0     0     0
[3,]     0     0     0     0
[4,]     0     0     0     0
```

Then function `table_n()` will compute the corresponding contingency tables with $n_{Y=1}$ and $n_{Y=0}$ stored in `n_type` along with the proportion effects $p_{X=1,Y=1}$ and $p_{X=1,Y=0}$ in `props`.

R Code Start Over Run Code

```
table_n <- function(n_type, props) {
  table <- matrix(0, 2, 2)
  table[, 2] <- (1 - matrix(props, 2, 1)) * n_type
  table[, 1] <- matrix(props, 2, 1) * n_type
  table
}
```

Finally

We compute the SEs by setting combination in `SE_log_OR`.

R Code Start Over Run Code

```
for (p in 1:dim(p_effects)[1]) {
  for (n in 1:length(n_cases)) {
    SE_log_OR[p, n] <- logOR_SE(table_n(
      matrix(c(n_controls, n_cases[n])),
      p_effects[p, ]
    ))
  }
}
round(SE_log_OR, 2)
```

```
     [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[,11] [,12] [,13] [,14]
[1,] 0.45 0.33 0.27 0.24 0.22 0.21 0.19 0.19 0.18  0.17
0.17  0.16  0.16  0.16
[2,] 0.42 0.31 0.26 0.23 0.21 0.20 0.19 0.18 0.17  0.17
0.16  0.16  0.15  0.15
[3,] 0.41 0.30 0.25 0.23 0.21 0.19 0.18 0.18 0.17  0.16
0.16  0.15  0.15  0.15
```

```
[4,] 0.42 0.31 0.26 0.23 0.21 0.20 0.19 0.18 0.17  0.17
0.16  0.16  0.15  0.15
     [,15] [,16] [,17] [,18] [,19] [,20] [,21] [,22] [,23]
[,24] [,25] [,26]
[1,]  0.15  0.15  0.15  0.15  0.14  0.14  0.14  0.14  0.14
0.14  0.14  0.13
[2,]  0.15  0.15  0.14  0.14  0.14  0.14  0.14  0.14  0.13
0.13  0.13  0.13
[3,]  0.15  0.14  0.14  0.14  0.14  0.14  0.14  0.13  0.13
0.13  0.13  0.13
[4,]  0.15  0.15  0.14  0.14  0.14  0.14  0.14  0.14  0.13
0.13  0.13  0.13
     [,27] [,28] [,29] [,30]
[1,]  0.13  0.13  0.13  0.13
[2,]  0.13  0.13  0.13  0.13
[3,]  0.13  0.13  0.13  0.13
[4,]  0.13  0.13  0.13  0.13
```

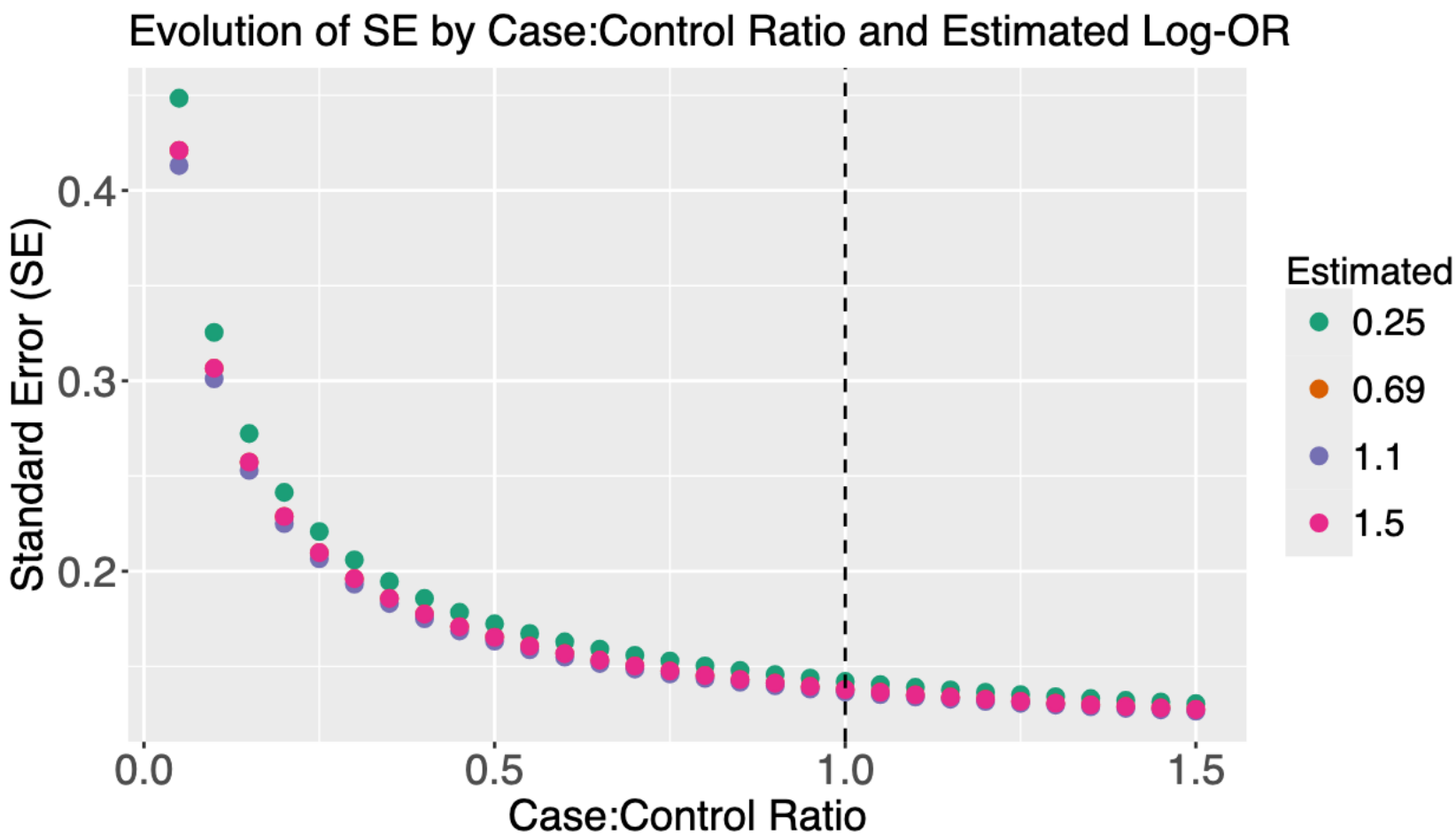
R Code Start Over Run Code

```r
# Wrangling our results before plotting
dat_plot <- data.frame(log_OR = log_OR, SE = SE_log_OR)
dat_plot <- melt(dat_plot, id.vars = c("log_OR"), value.name = "SE"
dat_plot$ratio <- n_cases[as.integer(
  dat_plot %>%
    separate(variable, into = c("pre", "post")) %>%
    pull("post")
)] / n_controls

# Plotting
CC_power_analysis <- ggplot(data = dat_plot, mapping = aes(x = rati
  geom_point(size = 3) +
  geom_vline(xintercept = 1, linetype = "dashed") +
  xlab("Case:Control Ratio") +
  ylab("Standard Error (SE)") +
  labs(color = "Estimated Log-OR") +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    legend.position = "right",
    legend.title = element_text(size = 16, margin = margin(r = 1, u
    legend.text = element_text(size = 16, margin = margin(r = 3, un
    legend.key.size = unit(1, 'cm')
  ) +
  ggtitle("Relationship of SE by Case:Control Ratio and Estimated Log-
  scale_color_brewer(palette = "Dark2")
```

R Code Start Over Run Code

```
CC_power_analysis
```



DSCI 554 - Experimentation and Causal Inference

What is the main takeaway from this plot?

- According to the SE behaviour, in populations where the event $Y = 1$ is rare, we get a more precise estimate of the $\log\text{-}\mathrm{OR}$ when we oversample cases and undersample controls.
- This is even more beneficial in situations where the estimated $\log\text{-}\mathrm{OR}$ is low (as shown in our colour scale).
- Ideally, we might keep the $\mathrm{Case{:}Control\ ratio}$ in 1 or slightly above in a CC scheme if $Y = 1$ is extremely rare.

6. Wrapping Up

- We have learned a new way to assess a given sampling scheme by relying on previously sampled data.
- This sampled data would allow building a proxy ground truth so we can run whole simulation studies for each sampling scheme. This proxy ground truth is assumed to be governed by a population regression model.
- Note there is a constant tool in our simulations: adding random noise.
- From the three sampling schemes we explored, CC can provide a boost in accuracy and precision.